# Robotics Lab 2025/2026

# Homework 2

***Control your robot***

Liccardo Antonio P38/136

GitHub Profile: https://github.com/LiccardoAntonio

For the purpose of streamlining the development of the code, the following adjustements and tools were used:

- Terminator was the terminal chosen, as the use of a custom layout can bring up and connect to the Docker ROS2 environment quickly;
- VSCode was started inside of the directory connected to the Docker `/src` directory of the workspace. The use of VSCode ROS2 extensions helped in correcting the syntax and keeping the workspace organized.

# Control your robot

1. To start development, the provided `ros2_kdl_node` and `ros2_kdl_node` package was cloned into the `/src` directory.

   (a) The specification 1.a requested to set some variables of the `ros2_kdl_node.cpp` as ros2 paramters. The declaration of these parameters is as follows:

   ```
   this->declare_parameter("traj_duration", 2.0);
   this->declare_parameter("acc_duration", 0.5);
   this->declare_parameter("total_time", 2.0);
   this->declare_parameter("trajectory_len", 160);
   this->declare_parameter("Kp", 0.1);
   ```

   These parameters were then saved into the relative variables via the `get_parameter()` function. The launch file has then been created in order to load parameter values from a yaml file and start the `ros2_kdl_node`.
   The `args_yaml` file is as follows:

   ```
   ros2_kdl_node:
     ros__parameters:

       traj_duration: 2.0
       acc_duration: 0.5
       total_time: 2.0
       trajectory_len: 160
       Kp: 5.0
       lambda: 0.1

       cmd_interface: "velocity"
       ctrl: "vision_ctrl"

       end_position: [0.2,0.0,0.3]

       action_server_mode: false
   ```

   The `ros2_kdl.launch.py` is as follows:

   ```
   from launch import LaunchDescription
   from launch.actions import DeclareLaunchArgument
   from launch.actions import IncludeLaunchDescription
   from launch.substitutions import Command, LaunchConfiguration,
       PathJoinSubstitution
   ```

```
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare
from launch.conditions import IfCondition, UnlessCondition
import os
from ament_index_python.packages import get_package_share_directory
from launch_ros.parameter_descriptions import ParameterValue

def generate_launch_description():
    kdl_node = Node(
        package='ros2_kdl_package',
        executable='ros2_kdl_node',
        name='ros2_kdl_node',
        output='screen',
        parameters=[PathJoinSubstitution([
            FindPackageShare('ros2_kdl_package'), 'config', 'args.yaml'])
        ],
    )

    return LaunchDescription([kdl_node,])
```

(b) The specification 1.b required to implement a new null space velocity controller that exploited the dexterity of the manipulator to keep the joints away from their limits.
This new control law is declared in the `kdl_control.h` (in the directory (`inside of /ros2_kdl_package/include`) coded within the realtive `kdl_control.cpp` body file (`inside of /ros2_kdl_package/src`).

The declaration inside of `kdl_control.h` is:

```
Eigen::VectorXd velocity_ctrl_null(KDL::JntArray &q_min,
                        KDL::JntArray &q_max,
                        double lambda,
                        KDL::JntArray &_qd,
                        double Kp,
                        Eigen::VectorXd &pos_error);
```

The parameters passed are necessary for the computation of the control law, and they are in order:

- minimum joint limit;
- maximum joint limit;
- lambda scaling factor;
- Kp proportional gain;
- current position and orientation error.

In order to pass the total position and orientation vector to the function call, a new memeber variable `Eigen::VectorXd total_error` was declared. At every iteration this vector is populated with the position and orientation errors within the private method of the node class:

```
total_error << error, o_error;
```

The body of the control law is the following:

```
Eigen::VectorXd KDLController::velocity_ctrl_null(KDL::JntArray &q_min,
                                                  KDL::JntArray &q_max,
```

```
                                                double lambda,
                                                KDL::JntArray &_qd,
                                                double Kp,
                                                Eigen::VectorXd &pos_error)
{
    unsigned int nj = robot_->getNrJnts();
    Eigen::VectorXd joint_velocity_cmd(nj);
    Eigen::VectorXd q0_dot(nj);

    //computation of q0_dot
    for(int i=0; i<nj; i++){
        double span = q_max(i) - q_min(i);
        q0_dot(i) = (1/lambda)*((span*span)/((q_max(i)-_qd(i))*(_qd(i)-q_min(i))
    ));
    }

    //computation of Jacobian Pseudoinverse
    Eigen::MatrixXd J = robot_->getEEJacobian().data;
    Eigen::MatrixXd J_pinv = J.completeOrthogonalDecomposition().pseudoInverse()
    ;

    //CONTROL LAW
    Eigen::VectorXd Kpe = (Kp*pos_error);
    printf("task calculated 1\n");

    Eigen::VectorXd task = J_pinv * Kpe;

    Eigen::MatrixXd I = Eigen::MatrixXd::Identity(nj, nj);
    //printf("identyty matrix defined\n");
    Eigen::MatrixXd N = I - J_pinv * J;
    //printf("N matrix calculated\n");
    Eigen::VectorXd null = N * q0_dot;
    //printf("null task calculated\n");

    joint_velocity_cmd = task + null;
    joint_velocity_cmd.cols());

    //printf("command calculated\n");

    return joint_velocity_cmd;
}
```

To keep the code here concise, a lot of debug code was left out. This function returns the angular velocities for each joint.
To let the user choose which controller to use, a new `ctrl` variable was created. When the selected command interface is set to velocity, this parameter is used to discern which controller to start between the pre-existing one and the newly implemented null space velocity controller:

```
if(cmd_interface_ == "velocity"){
    if(velocity_ctrl_type == "velocity_ctrl"){        //velocity controller
    previously implemented
        // Compute differential IK
        Vector6d cartvel; cartvel << p_.vel + Kp*error, o_error;
        joint_velocities_cmd_.data = pseudoinverse(robot_->getEEJacobian().data)
    *cartvel;
    }
    else if(velocity_ctrl_type == "velocity_ctrl_null"){ //null space controller
        joint_velocities_cmd_.data = controller_.velocity_ctrl_null(q_min, q_max
    , lambda, joint_positions_, Kp, total_error);
    }
}
```

In order to streamline the tuning process of this controller, both `ctrl` and `lambda` variables were declared as ros2 parameters and then inizialized in the `args.yaml` file.
The two controllers were then compared in regards of the performance by looking at the joint velocity command and joint states. The trajectory starts from the following joint position (in radiants) in which the manipulator is brought up in RViz2:

- joint_a1: 0.5
- joint_a2: -0.7854
- joint_a3: 0.0
- joint_a4: 1.3962
- joint_a5: 0.0
- joint_a6: 0.6109
- joint_a7: 0.0

The trajectory ends on the following end effector position (in meters):

- x = 0.2
- y = 0.0
- z = 0.3

The chosen gain `Kp` is 7.0, and the scaling factor `lambda` is 5.0.

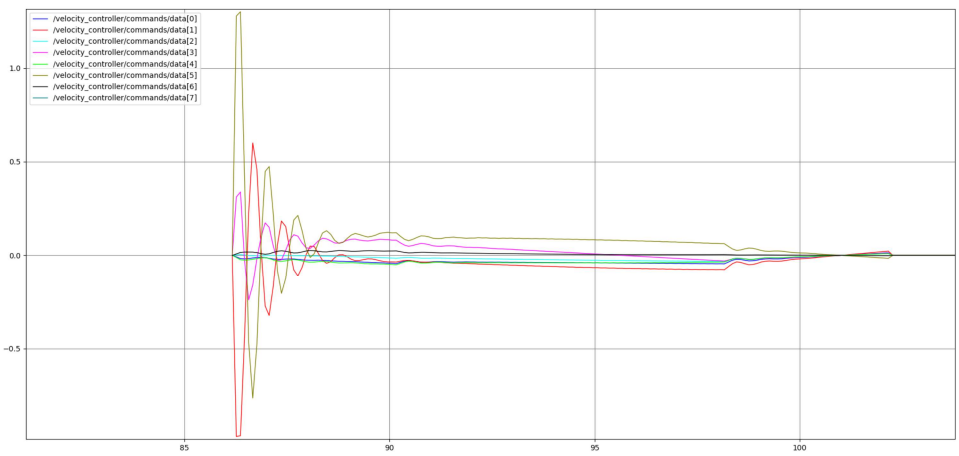


Figure 1: Plot of the joint velocities commands of the velocity controller

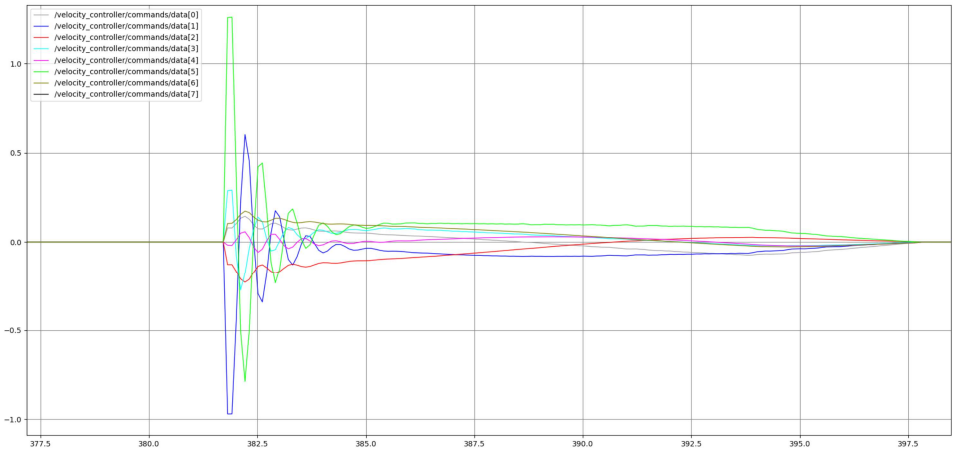


Figure 2: Plot of the joint velocities commands of the null space velocity controller

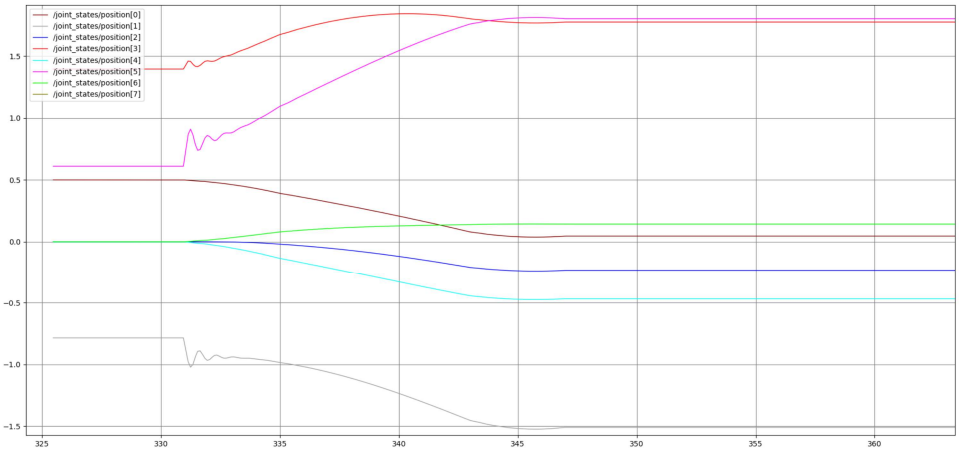


Figure 3: Plot of the joint states along the trajectory generated by the trajectory controller

The performance of the null space velocity controller is notably better, with lower peaks and shorter transient.

(c) The specification 1.c required the implementation of an action server wihtin the `ros2_kdl_node`. The implementative stategy adopted was to create a new boolean ros2 parameter `action_server_mode`. The code then uses this variable to discern if the nodes iterates the control routine through the already implemented timer callback or through the handler callback of the newly implemented action server.
To implement the server the following steps were taken:

i. The action interface was defined in the directory `/action` as `KdlFeedback.action`.

```
#Goal
bool start
---
#Result
bool completed
---
```

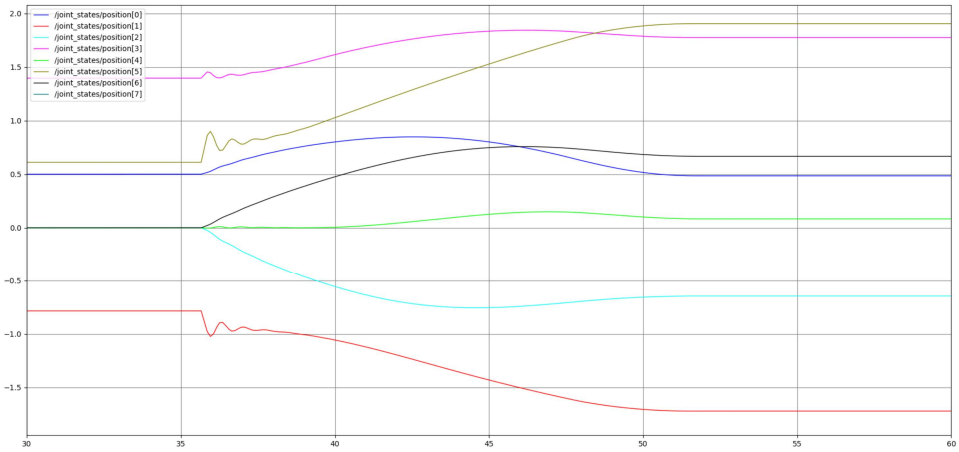


Figure 4: Plot of the joint states along the trajectory generated by the trajectory controller

```
#Feedback
float64[6] current_error
```

The goal is set as a boolean so that the action client would only send the start command. This implementative choise was made to keep consistency between the publisher mode and the action server mode, so that both of them would take the end position from the `args.yaml` file.

ii. To be able to include the interface both in the server script and client script, a new dipendency to `rosidl_default_generators` had to be specified both in the `CMakeList.txt` and in the package manifest. In the `CMakeList.txt` the dependendy has been specified in this way:

```
find_package(rosidl_default_generators REQUIRED)
```

In the package manifest the dependency has been specified in this way:

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>

<member_of_group>rosidl_interface_packages</member_of_group>
```

`rosidl_default_generators` is a tool that, starting from the action definition, can generate the adequate c++ header to include the action interface.

iii. The ros2 naming conventions for `.action` files requires to not use underscores, and using uppercase letters to separate words (matter of fact, the file is names `KdlFeedback.action`). When `rosidl_default_generators` generates the header, its name is changed so that uppercase letters became lowercase, and words are then separated with underscores. This results in the following header being included:

```
#include "ros2_kdl_package/action/kdl_feedback.hpp"
```

The header for the `rclcpp` action library was also included:

```
#include "rclcpp_action/rclcpp_action.hpp"
```

iv. In the public method two type aliases are declared:

```
using KdlFeedback = ros2_kdl_package::action::KdlFeedback;
using GoalHandleKdlFeedback = rclcpp_action::ServerGoalHandle<KdlFeedback>;
```

These aliases assess respectively the type of the KdlFeedback and the goal handler.

v. Any action server need three callbacks:

```
//handle goal callback
rclcpp_action::GoalResponse handle_goal(const rclcpp_action::GoalUUID &,
                                        std::shared_ptr<const KdlFeedback::
    Goal> goal)
{
    if (!goal->start) return rclcpp_action::GoalResponse::REJECT;
    return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE;
}

//handle cancel callback
rclcpp_action::CancelResponse handle_cancel(const std::shared_ptr<
    GoalHandleKdlFeedback>)
{
    return rclcpp_action::CancelResponse::ACCEPT;
}

//handle accepted callback
void handle_accepted(const std::shared_ptr<GoalHandleKdlFeedback>
    goal_handle)
{
    std::thread([this, goal_handle]() { execute(goal_handle); }).detach();
}
```

The `handle_accepted()` callback has the purpose al calling an `execute()` routine that then has the purpose of iterating the `cmd_publisher()` routine.

vi. The `execute()` routine is as follows:

```
void execute(const std::shared_ptr<GoalHandleKdlFeedback> goal_handle)
    {
    auto error_feedback = std::make_shared<KdlFeedback::Feedback>();
    auto result = std::make_shared<KdlFeedback::Result>();
    rclcpp::Rate rate(10);

    while(trajectory_executed == false)
    {
        if (goal_handle->is_canceling())
        {
        result->completed = false;
        goal_handle->canceled(result);
        return;
        }

        cmd_publisher();

        for(int i = 0; i<6;i++){
            error_feedback -> current_error[i] = total_error(i);
            RCLCPP_INFO(this->get_logger(),"%f",error_feedback ->
    current_error[i]);
```

```
        }

        goal_handle->publish_feedback(error_feedback);
        rate.sleep();
    }

    result->completed = true;
    goal_handle->succeed(result);
    }
```

In order to get the data for the `error_feedback` to publish fot the client to read, the `total_error` previously mentioned vector was made a class member for scoping purposes. The rate was set to 10Hz, which corresponds to the 100ms callback set by the default timer callback.

vii. The last step is to create the action server object. The script uses the `action_server_mode` that starts either the timer callback or the action server.
The following code is an example for the velocity command interface:

```
 if(cmd_interface_ == "velocity"){
    // Create cmd publisher
    cmdPublisher_ = this->create_publisher<FloatArray>("/velocity_controller
    /commands", 10);

    if(action_server_mode==false){
        timer_ = this->create_wall_timer(std::chrono::milliseconds(100),
                                    std::bind(&Iiwa_pub_sub::cmd_publisher,
    this));
    }else{
        this->action_server_ = rclcpp_action::create_server<KdlFeedback>(
    this,"kdl_feedback",
        std::bind(&Iiwa_pub_sub::handle_goal, this, std::placeholders::_1,
    std::placeholders::_2),
        std::bind(&Iiwa_pub_sub::handle_cancel, this, std::placeholders::_1)
    ,
        std::bind(&Iiwa_pub_sub::handle_accepted, this, std::placeholders::
    _1)
        );
    }
    // Set joint velocity commands
    for (long int i = 0; i < joint_velocities_.data.size(); ++i) {
        desired_commands_[i] = joint_velocities_(i);
    }
}
```

viii. To test the implementation a simple action client node (`ros2_kdl_action_client.cpp`) was created and added to the `CMakeList.txt`. It simply sends the start command, reads back the feedback and receives a stop command when the action has been executed.

2. The second project specification required to implement a vision-based task on the manipulator.

   (a) Under the directory `/ros2_iiwa/iiwa_description/gazebo/models` there can be found the `arucotag.png`

file relative to the aruco tag chosen. The file uri is then called in the `empty.world` file so that it can be spawned in the gazebo world. In order to be able to spawn the arugo tag entity the file URI needs to be exported to the environment via command:

```
export GZ_SIM_RESOURCE_PATH=$GZ_SIM_RESOURCE_PATH:~/ros2_ws/src/ros2_iiwa/
iiwa_description/gazebo/models/arucotag
```

The world can then be launched via:

```
ros2 launch iiwa_bringup iiwa.launch.py use_sim:=true gz_args:="-r ~/ros2_ws
/src/ros2_iiwa/iiwa_description/gazebo/worlds/empty.world"
```

In order to be able to detect the tag there needs to be mounted a camera on the manipulator. This has been done based on the example offered in the `ros2_sensors_and_actuators` (https://github.com/RoboticsLab2025/ros2_sensors_and_actuators). The following lines of code have been added to `iiwa.urdf.xacro` file to create the camera link and joint:

```
<link name="${prefix}camera_link">
    <visual>
        <geometry>
          <box size="0.0000010 0.00000003 0.00000003"/>
        </geometry>
        <material name="red"/>
    </visual>
</link>

<joint name="${prefix}camera_joint" type="fixed">
    <parent link="${prefix}tool0"/>
    <child link="camera_link"/>
    <origin xyz="0.0 0.0 0.0" rpy="3.14 -1.57 0"/>
</joint>
```

The following lines of code have then been added to define the camera sensor plugin and add the camera:

```
<gazebo>
    <plugin filename="gz-sim-sensors-system"
        name="gz::sim::systems::Sensors">
        <render_engine>ogre2</render_engine>
    </plugin>
</gazebo>

<gazebo reference="${prefix}camera_link">
    <sensor name="camera" type="camera">
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image>
        <width>320</width>
        <height>240</height>
      </image>
      <clip>
        <near>0.1</near>
        <far>100</far>
```

```
      </clip>
    </camera>
    <always_on>1</always_on>
    <update_rate>30</update_rate>
    <visualize>true</visualize>
    <topic>camera</topic>
    </sensor>
</gazebo>
```

To allow the recognition of the tag, the `aruco_ros` has been cloned into the workspace. In the gazebo world the tag is then adjusted to fit into the camera frame, and then the following command start the publishing of the marker pose relative to the camera frame to the `/aruco_single/pose` topic.

```
ros2 run aruco_ros single --ros-args \
  -p marker_id:=201 \
  -p marker_size:=0.1 \
  -p camera_frame:=link_7 \
  -p marker_frame:=aruco_marker_frame \
  -p reference_frame:=link_7 \
  -r /image:=/videocamera \
  -r /camera_info:=/camera_info
```

(b) For the vision control to be possible the `ros2_kdl_node` needs to be updated to include the subscriber to the `/aruco_single/pose` topic. If the`ctrl` parameter is set to `"vision_ctrl"` the node starts the subscriber:

```
if(velocity_ctrl_type == "vision_ctrl"){
    // Subscriber to aruco marker position
    arucoSubscriber_ = this->create_subscription<geometry_msgs::msg::PoseStamped
    >(
        "/aruco_single/pose", 10, std::bind(&Iiwa_pub_sub::aruco_pose_subscriber
    , this, std::placeholders::_1));
}
```

The relative function callback then saves into the `Eigen::Vector3d marker_pose` member variable the position data of the marker:

```
void aruco_pose_subscriber(const geometry_msgs::msg::PoseStamped& pose_msg_in){
    marker_pose << pose_msg_in.pose.position.x,
                   pose_msg_in.pose.position.y,
                   pose_msg_in.pose.position.z;
}
```

In the same way as the `velocity_ctrl_null` was implemented, in case the velocity command interface is selected, the `cmd_publisher` routine checks the `ctrl` parameter, and if the value is `"vision_ctrl"`, it calls the newly implemented `controller_.vision_ctrl()` function.

(c) To address the last specification 2.c, the following node object was created inside of the `ros2_kdl.launch.py` and returned to the launch configuration in order to start the ros-gazebo bridge.

```
gz_service_bridge = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',
```

```
        name='gz_service_bridge',
        arguments=[
            '/world/default/set_pose@ros_gz_interfaces/srv/SetEntityPose',
        ],
        output='screen'
    )
```

To set a new position for the marker the following command can be used:

```
ros2 service call /world/default/set_pose ros_gz_interfaces/srv/SetEntityPose "{
  entity: { name: 'arucotag', id: 0, type: 2 },
  pose: { position: {x: 0.7, y: 0.2, z: 2.1}, orientation: {w: 1.0, x: 0.0, y: 0.0,
    z: 0.0} }
}"
```